1P: NFA

```
class Nfa {
 static int nfa = 1;
static int flag = 0;
static void state1(char c)
{
   if (c == 'a')
      nfa = 2;
   else if (c == 'b' || c == 'c')
      nfa = 1;
   else
      flag = 1;
}
static void state2(char c)
{
   if (c == 'a')
      nfa = 3;
   else if (c == 'b' || c == 'c')
      nfa = 2;
   else
      flag = 1;
}
static void state3(char c)
{
   if (c == 'a')
      nfa = 1;
   else if (c == 'b' || c == 'c')
      nfa = 3;
   else
      flag = 1;
}
static void state4(char c)
{
   if (c == 'b')
      nfa = 5;
   else if (c == 'a' || c == 'c')
      nfa = 4;
   else
      flag = 1;
}
static void state5(char c)
{
   if (c == 'b')
      nfa = 6;
   else if (c == 'a' || c == 'c')
      nfa = 5;
   else
      flag = 1;
}
static void state6(char c)
{
   if (c == 'b')
      nfa = 4;
   else if (c == 'a' || c == 'c')
```

```
        nfa = 6;
    else
        flag = 1;
}
static void state7(char c)
{
    if (c == 'c')
        nfa = 8;
    else if (c == 'b' || c == 'a')
        nfa = 7;
    else
        flag = 1;
}
static void state8(char c)
{
    if (c == 'c')
        nfa = 9;
    else if (c == 'b' || c == 'a')
        nfa = 8;
    else
        flag = 1;
}
static void state9(char c)
{
    if (c == 'c')
        nfa = 7;
    else if (c == 'b' || c == 'a')
        nfa = 9;
    else
        flag = 1;
}
static boolean checkA(String s, int x)
{
    for (int i = 0; i < x; i++) {
        if (nfa == 1)
            state1(s.charAt(i));
        else if (nfa == 2)
            state2(s.charAt(i));
        else if (nfa == 3)
            state3(s.charAt(i));
    }
    if (nfa == 1) {
        return true;
    }
    else {
        nfa = 4;
    }
    return false;
}
static boolean checkB(String s, int x)
{
    for (int i = 0; i < x; i++) {
        if (nfa == 4)
            state4(s.charAt(i));
```

```
        else if (nfa == 5)
            state5(s.charAt(i));
        else if (nfa == 6)
            state6(s.charAt(i));
    }
    if (nfa == 4) {
        return true;
    }
    else {
        nfa = 7;
    }
    return false;
}
static boolean checkC(String s, int x)
{
    for (int i = 0; i < x; i++) {
        if (nfa == 7)
            state7(s.charAt(i));
        else if (nfa == 8)
            state8(s.charAt(i));
        else if (nfa == 9)
            state9(s.charAt(i));
    }
    if (nfa == 7) {
        return true;
    }
    return false;
}
public static void main (String[] args)
{
    String s = "aabca";
    int x = 5;
    if (checkA(s, x) || checkB(s, x) || checkC(s, x)) {
        System.out.println("STRING IS ACCEPTED");
    }
    else {
        if (flag == 0) {
            System.out.println("NOT ACCEPTED");
        }
        else {
            System.out.println("INPUT OUT OF BOUND");
        }
    }
}
}
```

2P: Min DFA

Disjointset.py:

```
class DisjointSet(object):
def __init__(self, items):
    self._disjoint_set = list()
    if items:
        for item in set(items):
            self._disjoint_set.append([item])
def _get_index(self, item):
```

```
    for s in self._disjoint_set:
        for _item in s:
            if _item == item:
                return self._disjoint_set.index(s)
    return None
def find(self, item):
    for s in self._disjoint_set:
        if item in s:
            return s
    return None
def find_set(self, item):
    s = self._get_index(item)
    return s + 1 if s is not None else None
def union(self, item1, item2):
    i = self._get_index(item1)
    j = self._get_index(item2)
    if i != j:
        self._disjoint_set[i] += self._disjoint_set[j]
        del self._disjoint_set[j]
def get(self):
    return self._disjoint_set
```

DFA.py:

```
from collections import defaultdict
from disjoint_set import DisjointSet
class DFA(object):
def __init__(self, states_or_filename, terminals=None,
             start_state=None, transitions=None, final_states=None):
    # if values in graph file
    if terminals is None:
        self._get_graph_from_file(states_or_filename)
    # if manual values
    else:
        assert isinstance(states_or_filename, (list, tuple))
        self.states = states_or_filename
        assert isinstance(terminals, (list, tuple))
        self.terminals = terminals
        assert isinstance(start_state, str)
        self.start_state = start_state
        assert isinstance(transitions, dict)
        self.transitions = transitions
        assert isinstance(final_states, (list, tuple))
        self.final_states = final_states
def _remove_unreachable_states(self):
    """
    Removes states that are unreachable from the start state
    """
    g = defaultdict(list)
    for k, v in self.transitions.items():
        g[k[0]].append(v)
    # DFS
    stack = [self.start_state]
    reachable_states = set()
    while stack:
        state = stack.pop()
```

```
        if state not in reachable_states:
            stack += g[state]
            reachable_states.add(state)
    self.states = [s for s in self.states if s in reachable_states]
    self.final_states = [s for s in self.final_states
                         if s in reachable_states]
    self.transitions = {
        k: v for k, v in self.transitions.items()
        if k[0] in reachable_states
    }
def minimize(self):
    self._remove_unreachable_states()
    def order_tuple(a, b):
        return (a, b) if a < b else (b, a)
    table = {}
    sorted_states = sorted(self.states)
    # initialize table
    for i, item in enumerate(sorted_states):
        for item_2 in sorted_states[i + 1:]:
            table[(item, item_2)] = (
                (item in self.final_states) !=
                (item_2 in self.final_states)
            )
    flag = True
    # table filling method
    while flag:
        flag = False
        for i, item in enumerate(sorted_states):
            for item_2 in sorted_states[i + 1:]:
                if table[(item, item_2)]:
                    continue
                for w in self.terminals:
                    t1 = self.transitions.get((item, w), None)
                    t2 = self.transitions.get((item_2, w), None)
                    if t1 is not None and t2 is not None and t1 != t2:
                        marked = table[order_tuple(t1, t2)]
                        flag = flag or marked
                        table[(item, item_2)] = marked
                        if marked:
                            break
    d = DisjointSet(self.states)
    # form new states
    for k, v in table.items():
        if not v:
            d.union(k[0], k[1])
    self.states = [str(x) for x in range(1, 1 + len(d.get()))]
    new_final_states = []
    self.start_state = str(d.find_set(self.start_state))
    for s in d.get():
        for item in s:
            if item in self.final_states:
                new_final_states.append(str(d.find_set(item)))
                break
    self.transitions = {
```

```
        (str(d.find_set(k[0])), k[1]): str(d.find_set(v))
        for k, v in self.transitions.items()
    }
    self.final_states = new_final_states
def show(self):
    temp = defaultdict(list)
    for k, v in self.transitions.items():
        temp[(k[0], v)].append(k[1])
    return temp
def __str__(self):
    """
    String representation
    """
    num_of_state = len(self.states)
    start_state = self.start_state
    num_of_final = len(self.final_states)
    return '{} states. {} final states. start state - {}'.format(
        num_of_state, num_of_final, start_state
    )
def _get_graph_from_file(self, filename):
    """
    Load the graph from file
    """
    with open(filename, 'r') as f:
        try:
            lines = f.readlines()
            states, terminals, start_state, final_states = lines[:4]
            if states:
                self.states = states[:-1].split()
            else:
                raise Exception("Invalid file format: cannot read states")
            if terminals:
                self.terminals = terminals[:-1].split()
            else:
                raise Exception("Invalid file format: cannot read terminals")
            if start_state:
                self.start_state = start_state[:-1]
            else:
                raise Exception("Invalid file format: cannot read start state")
            if final_states:
                self.final_states = final_states[:-1].split()
            else:
                raise Exception("Invalid file format: cannot read final states")
            lines = lines[4:]
            self.transitions = {}
            for line in lines:
                current_state, terminal, next_state = line[:-1].split()
                self.transitions[(current_state, terminal)] = next_state
        except Exception as e:
            print("ERROR:", e)
if __name__ == "__main__":
filename = "graph"
dfa = DFA(filename)
# initial DFA
```

```
x = dict(dfa.show())
for key, value in x.items():
    print(key, ":", value)
print(dfa, "\n")
dfa.minimize()
# minimized DFA
x = dict(dfa.show())
for key, value in x.items():
    print(key, ":", value)
print(dfa)
graph:
1 2 3 4 5
a b
1
1 5
1 a 3
1 b 2
2 b 1
2 a 4
3 b 4
3 a 5
4 a 4
4 b 4
5 a 3
5 b 2
```

3P: DFA for the given Language

GFG.java:

```
// JAVA Program to DFA that accept Strings
// which starts and end with 'a' over input(a, b)
import java.util.*;
class GFG {
public static void main(String[] args) {
    // for producing different random numbers every time
    Random r = new Random();
    // random length of String from 1 - 16
    int max = 1 + r.nextInt() * 10 % 15;
    int i = 0;
    // generating random String and processing it
    while (i < max) {
        // producing random character over input alphabet (a, b)
        char c = (char) ('a' + r.nextInt() * 10 % 2);
        System.out.print(c + " ");
        i++;
        // first character is 'a'
        if (c == 'a') {
            // if only one character
            if (i == max)
                System.out.print("YES\n");
            while (i < max) {
                c = (char) ('a' + r.nextInt() * 10 % 2);
                System.out.print(c + " ");
                i++;
                // last character check
                if (c == 'a' && i == max) {
```

```
                System.out.print("\nYES\n");
            }
            else if (i == max) {
                System.out.print("\nNO\n");
            }
        }
    }
    // first character is 'b'
    else {
        while (i < max) {
            c = (char) ('a' + r.nextInt() * 10 % 2);
            System.out.print(c + " ");
            i++;
        }
        System.out.print("\nNO\n");
    }
  }
}
}
```

GFG2.java:

```
// Java program to implement DFA
// L = { a^n b^m ; n mod 2 = 0, m >= 1 }
class GFG2 {
// stores current DFA state
static int dfa = 0;
// Start state Q0
static void start(char c) {
  if (c == 'a')
    dfa = 1;
  else if (c == 'b')
    dfa = 3;
  else
    dfa = -1;
}
// State Q1
static void state1(char c) {
  if (c == 'a')
    dfa = 2;
  else if (c == 'b')
    dfa = 4;
  else
    dfa = -1;
}
// State Q2
static void state2(char c) {
  if (c == 'b')
    dfa = 3;
  else if (c == 'a')
    dfa = 1;
  else
    dfa = -1;
}
// State Q3
static void state3(char c) {
```

```
    if (c == 'b')
        dfa = 3;
    else if (c == 'a')
        dfa = 4;
    else
        dfa = -1;
}
// Dead state Q4
static void state4(char c) {
    dfa = -1;
}
// DFA checker
static int isAccepted(char str[]) {
    int len = str.length;
    for (int i = 0; i < len; i++) {
        if (dfa == 0)
            start(str[i]);
        else if (dfa == 1)
            state1(str[i]);
        else if (dfa == 2)
            state2(str[i]);
        else if (dfa == 3)
            state3(str[i]);
        else if (dfa == 4)
            state4(str[i]);
        else
            return 0;
    }
    if (dfa == 3)
        return 1;
    else
        return 0;
}
// Driver code
public static void main(String[] args) {
    char str[] = "aaaaaabbbb".toCharArray();
    if (isAccepted(str) == 1)
        System.out.println("ACCEPTED");
    else
        System.out.println("NOT ACCEPTED");
}
}
```

4P: C program to check syntax of 'for' loop

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
// array to copy first three characters
char arr[3];
void isCorrect(char *str)
{
// counters
int semicolon = 0, bracket1 = 0, bracket2 = 0, flag = 0;
int i;
// copy first 3 characters
```

```
for (i = 0; i < 3; i++)
    arr[i] = str[i];
// check "for" keyword
if (strcmp(arr, "for") != 0)
{
    printf("Error in for keyword usage");
    return;
}
// check syntax characters
while (i != strlen(str))
{
    char ch = str[i++];
    if (ch == '(')
        bracket1++;
    else if (ch == ')')
        bracket2++;
    else if (ch == ';')
        semicolon++;
}
// check semicolons
if (semicolon != 2)
{
    printf("\nSemicolon Error");
    flag++;
}
// check closing parenthesis
else if (str[strlen(str) - 1] != ')')
{
    printf("\nClosing parenthesis absent at end");
    flag++;
}
// check opening parenthesis
else if (str[3] == ' ' && str[4] != '(')
{
    printf("\nOpening parenthesis absent after for keyword");
    flag++;
}
// check parentheses count
else if (bracket1 != 1 || bracket2 != 1 || bracket1 != bracket2)
{
    printf("\nParentheses Count Error");
    flag++;
}
// no error
if (flag == 0)
    printf("\nNo error");
}
int main()
{
char str1[100] = "for (i = 10; i < 20; i++)";
isCorrect(str1);
char str2[100] = "for i = 10; i < 20; i++)";
isCorrect(str2);
return 0;
```

}

5P: Construct NFA using given regular expression.

nfa.py:

```
from nfa_utils import *
import sys
import time
class NFA:
"""Class representing a non-deterministic finite automaton"""
def __init__(self):
    """Creates a blank NFA"""
    # all NFAs have a single initial state by default
    self.alphabet = set()
    self.states = {0}
    self.transition_function = {}
    self.accept_states = set()
    # set of states that the NFA is currently in
    self.in_states = {0}
def add_state(self, state, accepts=False):
    self.states.add(state)
    if accepts:
        self.accept_states.add(state)
def add_transition(self, from_state, symbol, to_states):
    self.transition_function[(from_state, symbol)] = to_states
    if symbol != "":
        self.alphabet.add(symbol)
def is_accepting(self):
    """
    Accepts if we are in ANY accept states
    i.e., if in_states and accept_states share any states
    """
    return len(self.in_states & self.accept_states) > 0
def __str__(self):
    return (
        "NFA:\n"
        "Alphabet: {}\n"
        "States: {}\n"
        "Transition Function:\n{}\n"
        "Accept States: {}\n"
        "In states: {}\n"
        "Accepting: {}\n"
    ).format(
        self.alphabet,
        self.states,
        self.transition_function,
        self.accept_states,
        self.in_states,
        "Yes" if self.is_accepting() else "No",
    )
if __name__ == "__main__":
# regular expression string to compare against provided input
regex = None
regex_nfa = None
# read user input
line_read = input("Enter the regex pattern \n> ")
```

```
# lowercase copy for case-insensitive comparisons
regex = line_read.lower()
print("Given Regex pattern:", regex, "\n")
print("Building NFA...")
start_time = time.time()
# convert regex string into an NFA object
regex_nfa = get_regex_nfa(regex)
finish_time = time.time()
ms_taken = (finish_time - start_time) * 1000
print("\nBuilt NFA in {:.3f} ms.\n".format(ms_taken))
print(regex_nfa)
```

nfa_utils.py:

```
from nfa import NFA
import copy
def get_single_symbol_regex(symbol):
"""Returns an NFA that recognizes a single symbol"""
nfa = NFA()
nfa.add_state(1, True)
nfa.add_transition(0, symbol, {1})
return nfa
def shift(nfa, inc):
"""
Increases the value of all states (including accept states and transition
function) of a given NFA by a given value.
"""
# update NFA states
new_states = set()
for state in nfa.states:
    new_states.add(state + inc)
nfa.states = new_states
# update accept states
new_accept_states = set()
for state in nfa.accept_states:
    new_accept_states.add(state + inc)
nfa.accept_states = new_accept_states
# update transition function
new_transition_function = {}
for pair in nfa.transition_function:
    to_set = nfa.transition_function[pair]
    new_to_set = set()
    for state in to_set:
        new_to_set.add(state + inc)
    new_key = (pair[0] + inc, pair[1])
    new_transition_function[new_key] = new_to_set
nfa.transition_function = new_transition_function
def merge(a, b):
"""Merges two NFAs"""
a.accept_states = b.accept_states
a.states |= b.states
a.transition_function.update(b.transition_function)
a.alphabet |= b.alphabet
def get_concat(a, b):
"""Concatenation operator (.)"""
add = max(a.states)
```

```
shift(b, add)
merge(a, b)
return a
def get_union(a, b):
"""Union operator (|)"""
nfa = NFA()
# clear accept states
a.accept_states = set()
b.accept_states = set()
# merge a
shift(a, 1)
merge(nfa, a)
# merge b
shift(b, max(nfa.states) + 1)
merge(nfa, b)
# epsilon transitions
nfa.add_transition(0, "", {1, min(b.states)})
# new accept state
new_accept = max(nfa.states) + 1
nfa.add_state(new_accept, True)
nfa.add_transition(max(a.states), "", {new_accept})
nfa.add_transition(max(b.states), "", {new_accept})
return nfa
def get_kleene_star_nfa(nfa):
"""
Kleene star (*)
"""
# clear old accept states
nfa.accept_states = set()
# shift and add new start state
shift(nfa, 1)
nfa.add_state(0)
last_state = max(nfa.states)
new_accept = last_state + 1
nfa.add_state(new_accept, True)
nfa.add_transition(last_state, "", {new_accept})
# epsilon transitions
nfa.add_transition(0, "", {1, new_accept})
nfa.add_transition(last_state, "", {0})
return nfa
def get_one_or_more_of_nfa(nfa):
"""Plus operator (+)"""
return get_concat(
    copy.deepcopy(nfa),
    get_kleene_star_nfa(nfa)
)
def get_zero_or_one_of_nfa(nfa):
"""Question mark operator (?)"""
return get_union(get_single_symbol_regex(""), nfa)
# ================ MAIN REGEX BUILDER =================
def get_regex_nfa(regex, indent=""):
"""Recursively builds an NFA from regex"""
print(f"{indent}{indent}({regex})")
indent += " " * 2
```

```
# UNION |
bar_pos = regex.find("|")
if bar_pos != -1:
    return get_union(
        get_regex_nfa(regex[:bar_pos], indent),
        get_regex_nfa(regex[bar_pos + 1:], indent)
    )
# CONCAT .
dot_pos = regex.find(".")
if dot_pos != -1:
    return get_concat(
        get_regex_nfa(regex[:dot_pos], indent),
        get_regex_nfa(regex[dot_pos + 1:], indent)
    )
# KLEENE *
star_pos = regex.find("*")
if star_pos != -1:
    star_part = regex[:star_pos]
    trailing_part = regex[star_pos + 1:]
    kleene_nfa = get_kleene_star_nfa(
        get_regex_nfa(star_part, indent)
    )
    if len(trailing_part) > 0:
        return get_concat(
            kleene_nfa,
            get_regex_nfa(trailing_part, indent)
        )
    return kleene_nfa
# PLUS +
plus_pos = regex.find("+")
if plus_pos != -1:
    plus_part = regex[:plus_pos]
    trailing_part = regex[plus_pos + 1:]
    plus_nfa = get_one_or_more_of_nfa(
        get_regex_nfa(plus_part, indent)
    )
    if len(trailing_part) > 0:
        return get_concat(
            plus_nfa,
            get_regex_nfa(trailing_part, indent)
        )
    return plus_nfa
# QUESTION MARK ?
qmark_pos = regex.find("?")
if qmark_pos != -1:
    leading_part = regex[:qmark_pos]
    trailing_part = regex[qmark_pos + 1:]
    zero_or_one = get_zero_or_one_of_nfa(
        get_regex_nfa(leading_part, indent)
    )
    if len(trailing_part) > 0:
        return get_concat(
            zero_or_one,
            get_regex_nfa(trailing_part, indent)
```

```
        )
    return zero_or_one
# BASE CASES
if len(regex) == 0:
    return NFA()
elif len(regex) == 1:
    return get_single_symbol_regex(regex)
else:
    return get_concat(
        get_regex_nfa(regex[0], indent),
        get_regex_nfa(regex[1:], indent)
    )
```

6P: generate the DAG for the input arithmetic expression.

```
class Graph:
# Constructor
def __init__(self, edges, n):
    # A list of lists to represent an adjacency list
    self.adjList = [[] for _ in range(n)]
    # add edges to the directed graph
    for (src, dest) in edges:
        self.adjList[src].append(dest)
# Perform DFS on the graph and set the departure time of all vertices of the graph
def DFS(graph, v, discovered, departure, time):
# mark the current node as discovered
discovered[v] = True
# do for every edge (v, u)
for u in graph.adjList[v]:
    # if `u` is not yet discovered
    if not discovered[u]:
        time = DFS(graph, u, discovered, departure, time)
# ready to backtrack
# set departure time of vertex `v`
departure[v] = time
time = time + 1
return time
# Returns true if the given directed graph is DAG
def isDAG(graph, n):
# keep track of whether a vertex is discovered or not
discovered = [False] * n
# keep track of the departure time of a vertex in DFS
departure = [None] * n
time = 0
# Perform DFS traversal from all undiscovered vertices
# to visit all connected components of a graph
for i in range(n):
    if not discovered[i]:
        time = DFS(graph, i, discovered, departure, time)
# check if the given directed graph is DAG or not
for u in range(n):
    # check if (u, v) forms a back-edge.
    for v in graph.adjList[u]:
        # If the departure time of vertex `v` is greater than equal
        # to the departure time of `u`, they form a back edge.
        # Note that `departure[u]` will be equal to `departure[v]`
```

```
        # only if `u = v`, i.e., vertex contain an edge to itself
        if departure[u] <= departure[v]:
            return False
# no back edges
return True
if __name__ == '__main__':
# List of graph edges as per the above diagram
edges = [(0, 1), (0, 3), (1, 2), (1, 3), (3, 2), (3, 4), (3, 0), (5, 6), (6, 3)]
# total number of nodes in the graph (labelled from 0 to 6)
n = 7
# build a graph from the given edges
graph = Graph(edges, n)
# check if the given directed graph is DAG or not
if isDAG(graph, n):
    print('The graph is a DAG')
else:
    print('The graph is not a DAG')
```

7P: loop unrolling and loop splitting

```
#include <stdio.h>
// Demonstration 1: Loop Unrolling
void demonstrate_loop_unrolling() {
printf("--- Loop Unrolling Demonstration ---\n");
int arr[8] = {5, 10, 15, 20, 25, 30, 35, 40};
int sum_standard = 0;
int sum_unrolled = 0;
// 1. Standard Loop execution
// The loop control (i < 8) and increment (i++) happen 8 times.
for (int i = 0; i < 8; i++) {
    sum_standard += arr[i];
}
printf("Standard Loop Sum: %d\n", sum_standard);
// 2. Unrolled Loop execution (Unroll factor of 4)
// The loop control and increment only happen 2 times.
for (int i = 0; i < 8; i += 4) {
    sum_unrolled += arr[i];
    sum_unrolled += arr[i+1];
    sum_unrolled += arr[i+2];
    sum_unrolled += arr[i+3];
}
printf("Unrolled Loop Sum: %d\n", sum_unrolled);
}
// ---------------------------------------------------------
// Demonstration 2: Loop Splitting (Peeling)
// ---------------------------------------------------------
void demonstrate_loop_splitting() {
printf("\n--- Loop Splitting (Peeling) Demonstration ---\n");
// Array of size 11 so x[10] is valid
int x[11] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 99};
int y_standard[10];
int y_split[10];
int p = 10;
// 1. Standard Loop with complex dependency
// 'p' changes behavior after the very first iteration.
for (int i = 0; i < 10; ++i) {
```

```
    y_standard[i] = x[i] + x[p];
    p = i;
}
// 2. Split (Peeled) Loop
// The problematic first iteration (where p=10) is performed separately.
y_split[0] = x[0] + x[10];
// The remaining iterations proceed without needing the 'p' variable.
for (int i = 1; i < 10; ++i) {
    y_split[i] = x[i] + x[i-1];
}
// Verify both methods produce the exact same results
printf("Verification (First 5 elements):\n");
for (int i = 0; i < 5; i++) {
    printf("Index %d -> Standard: %d | Split: %d\n", i, y_standard[i], y_split[i]);
}
}
int main() {
// Execute both demonstrations
demonstrate_loop_unrolling();
demonstrate_loop_splitting();
return 0;
}
```